El núcleo: Introducción

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

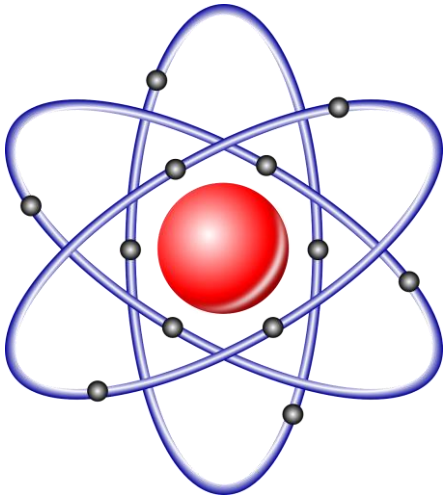
Contexto

- Los SO se ejecutan en **entornos hostiles** donde existen muchas amenazas (virus, malware, recursos limitados, …)

- Los navegadores web pueden ejecutar código JavaScript.
  - Si el navegador ofrece protección, un código JavaScript malicioso podría tomar el control del navegador, accediendo a historial de navegación, contraseñas, firmas, etc.
- Algunas Apps ejecutan código de terceros.
  - (+) La App permite ampliar su funcionalidad
  - (-) Riesgos de seguridad. Requiere de protección frente a códigos externos.

¿Qué es un núcleo?

- El **núcleo** (o kernel) es el **componente central** del SO.
- Es el encargado de gestionar los **recursos** del hardware y proporciona una **interfaz** para que las Apps interactúen con el hardware.
- Responsable de la **protección** del SO.



- En este tema veremos los siguientes puntos:
  - Introducción
  - El concepto de proceso
  - Modo dual de operación
  - Transferencia de control de seguridad entre usuario y núcleo
  - Estructura de un SO
  - Conclusiones

¿De qué aspectos se encarga el núcleo?

- **Fiabilidad**: Un error en la programación en una App (o malware) no debería afectar al resto de Apps. El SO debería funcionar correctamente independientemente de las Apps en ejecución.
- **Seguridad**: Es necesario limitar las acciones que realizan las aplicaciones. P.ej. Impedir que una App escriba directamente datos a disco o modifique el propio código del SO.
- **Privacidad**: En SO multiusuario, cada usuario (y sus Apps) solo podrá acceder a los datos a los cuales tiene permiso.
- **Eficiencia**: Las Apps no deberían consumir todos los recursos de la máquina (CPU, memoria) en detrimento de otras Apps. El SO debería repartir los recursos.

¿Cómo lo consigue?

- El núcleo:
  - es la pieza de **confianza** del software para comunicarse con el hardware.
  - tiene **acceso total** a las capacidades del hardware.
  - se ejecuta directamente en el procesador con **permisos ilimitados**.

- Las procesos (Apps de usuarios)
  - no son de confianza para el SO,
  - se ejecutan en un **entorno restringido**, con acceso limitado a la capacidad real del hardware.
  - se ejecutan en el procesador con ciertas **limitaciones** en operaciones potencialmente peligrosas (p.ej. escritura de datos en disco) gracias a la colaboración del hardware.

Acceso a recursos

- La gestión del acceso de un **proceso** al hardware está controlado por el núcleo.

- Un proceso (App en ejecución) necesita **autorización del núcleo** para acceder a: (i) una determinada posición de memoria, (ii) para escribir a disco, (iii) modificar la configuración de una máquina, etc.

Preguntas que responderemos en este tema

- ¿Qué se diferencia entre un **proceso** y un **programa**?
- ¿Como ejecuta un SO un proceso? Que hardware hace falta para ejecutarlo de forma eficiente un proceso en modo restringido? **Modo dual de operación**
- ¿Como es posible pasar la frontera entre procesos no fiables y el núcleo fiable? **Transferencia de control de seguridad usuario-núcleo**

Gracias

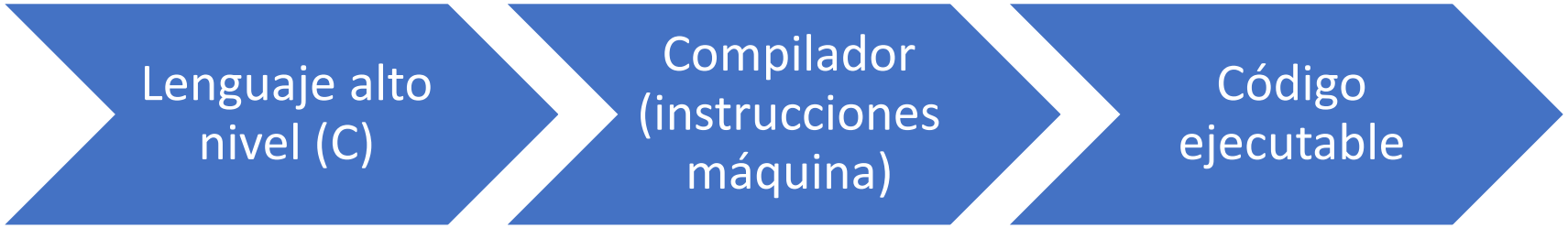
El núcleo: el proceso

Sistemas Operativos I

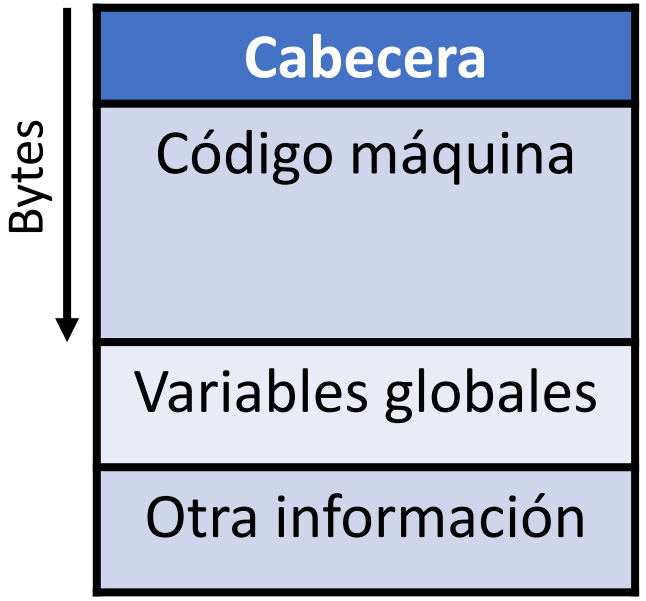
Oliver Díaz

Grau d'Enginyeria Informàtica

Concepto de proceso



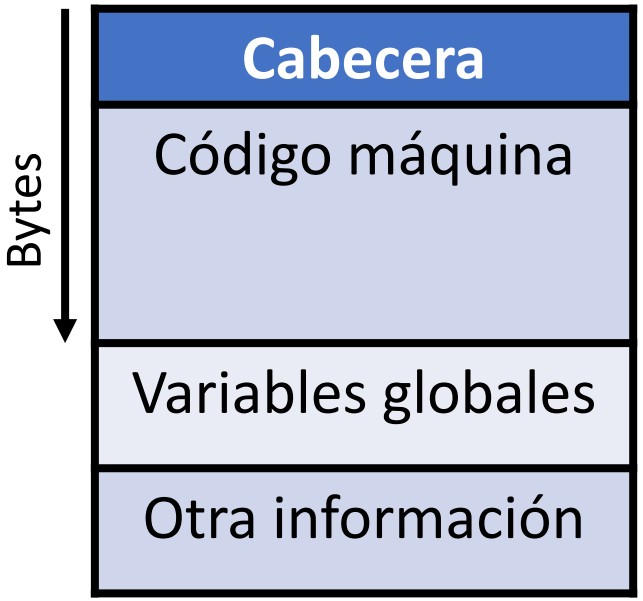
- El código ejecutable tiene una estructura interna conocida por el SO.



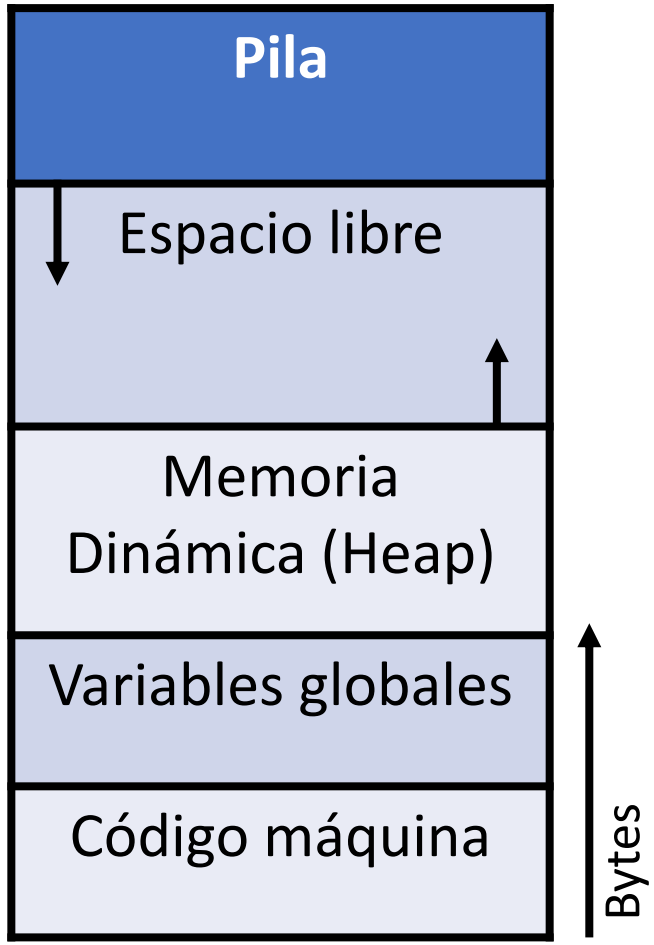
Estructura de un programa

¿Qué pasa al ejecutar un programa?

- SO llama a la función *main* (el proceso se esta ejecutando)
- SO copia las instrucciones máquina de disco a memoria
- Crea la pila
- Crea zona memoria dinámica



Estructura de un programa



Estructura de un proceso en memoria

Definamos el proceso

- Un proceso es **una instancia de un programa** (de la misma forma que un objeto es una instancia de una clase en POO)
- El compilador es un programa y al ejecutarlo, el SO crea el proceso correspondiente. Lo mismo pasa con el intérprete de Python, Java, navegador web, etc.
- Podemos ejecutar múltiples instancias de un mismo programa. El SO se encarga de cargar a memoria múltiples copias de las instrucciones del programa, así como crear múltiples copias de la pila y zona de memoria dinámica.
- La creación de una **pila** y zona de **memoria dinámica** implica asignar los permisos necesarios a aquella zona para que el proceso pueda leer y/o escribir.

Veamos un ejemplo

- **Región global**
  - Variables globales (*global*)
- **Región pila**
  - Variables locales de la función (*a,b*)
  - Histórico llamadas a sistema
- **Región de memoria dinámica**
  - *vector* de tamaño 1000

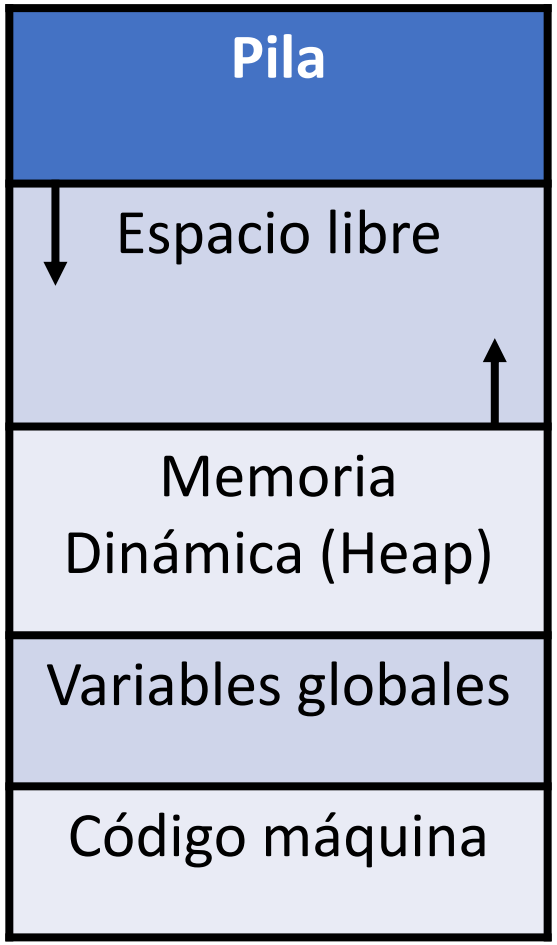
```c
int global;

int func(int a)
{
  int b;
  // Processament
}

void main(void)
{
  char str[10];
  int local, *vector;

  global = local = 0;

  // Processament
  func(2);
  // Reservar memoria
  vector = malloc(1000);
}
```



Estructura de un proceso en memoria

Gracias

El núcleo: Modo dual de operación

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Contexto

- Supongamos que ejecutamos un programa
- El SO copia las instrucciones máquina a ejecutar a memoria
- Se comienza a ejecutar el proceso a través de la función principal (*main*)
- La CPU comienza a ejecutar las instrucciones que encuentra: multiplicaciones, escritura y lectura de posiciones de memoria, etc.

¿Cómo asegura el SO que las instrucciones de un proceso no interfieren con los otros procesos o el núcleo? Recordad que el SO debe asegurar la protección entre procesos.

Modo de operación dual

- El mecanismo actual se basa en que el **hardware se puede ejecutar en dos modos**, cada uno con diferentes niveles de privilegio.
- El modo dual de operación
  - En **modo núcleo** (o kernel) el procesador puede ejecutar cualquier instrucción.
  - En **modo usuario** el procesador comprueba cada instrucción antes de ejecutarla.
- El núcleo se ejecuta en modo núcleo, mientras que los procesos en modo usuario.
- Si el hardware detecta que un proceso hace una operación inválida, se avisa al núcleo de esta infracción.

Modo de operación dual

- ¿Qué hardware es necesario para que el núcleo pueda proteger los procesos entre sí?
  - **Instrucciones privilegiadas:** Todas las instrucciones potencialmente peligrosas están prohibidas en modo usuario.
  - **Protección de memoria**: Todos los accesos a memoria física de un proceso fuera del espacio que se le ha asignado están prohibidos en modo usuario.
  - **Interrupciones de temporizador**: Independientemente de lo que haga un proceso, el núcleo del proceso debe tomar el control de forma periódica de la máquina para realizar "gestiones".
- Además, el hardware debe proveer de un mecanismo para pasar de forma segura de modo núcleo a modo usuario ya la inversa.

Modo de operación dual

- ¿Qué hardware es necesario para que el núcleo pueda proteger los procesos entre sí?
  - **Instrucciones privilegiadas:** Todas las instrucciones potencialmente peligrosas están prohibidas en modo usuario.
  - **Protección de memoria**: Todos los accesos a memoria física de un proceso fuera del espacio que se le ha asignado están prohibidos en modo usuario.
  - **Interrupciones de temporizador**: Independientemente de lo que haga un proceso, el núcleo del proceso debe tomar el control de forma periódica de la máquina para realizar "gestiones".
- Además, el hardware debe proveer de un mecanismo para pasar de forma segura de modo núcleo a modo usuario ya la inversa.

Instrucciones privilegiadas

Los **procesos de usuario** que pueden utilizar solo un **subconjunto de todas las instrucciones disponibles** (modo de usuario de ejecución), mientras que el **núcleo** tiene a su disposición **toda la potencia de la máquina** (modo núcleo de ejecución).

¿Qué instrucciones se deberían restringir?

Instrucciones privilegiadas

- Un proceso de usuario no debe ser capaz de modificar, entre otras cosas, su nivel de privilegio de ejecución. Tampoco debe poder modificar las interrupciones del hardware, ni cambiar la región de memoria a la que puede acceder, etc.

- Un proceso puede obtener privilegios de forma indirecta mediante una **llamada a sistema**: una llamada que transfiere el control al SO para realizar la operación solicitada por el proceso (i.e. escribir en disco).

Instrucciones privilegiadas

¿Qué pasa si un proceso de usuario ejecuta una instrucción no autorizada?

- Este evento provoca una **excepción de procesador**. La excepción provoca que el hardware transfiera de forma inmediata el control a una determinada función del núcleo, que puede tomar las acciones necesarias.

- Típicamente, al producirse una excepción por una instrucción inválida, esta función decide detener el proceso que debe provocado la excepción (se "mata").

Modo de operación dual

- ¿Qué hardware es necesario para que el núcleo pueda proteger los procesos entre sí?
  - **Instrucciones privilegiadas:** Todas las instrucciones potencialmente peligrosas están prohibidas en modo usuario.
  - **Protección de memoria**: Todos los accesos a memoria física de un proceso fuera del espacio que se le ha asignado están prohibidos en modo usuario.
  - **Interrupciones de temporizador**: Independientemente de lo que haga un proceso, el núcleo del proceso debe tomar el control de forma periódica de la máquina para realizar "gestiones".
- Además, el hardware debe proveer de un mecanismo para pasar de forma segura de modo núcleo a modo usuario ya la inversa.

Protección de memoria

Para ejecutar un proceso (de usuario), es necesario que

- El proceso resida en memoria física para que pueda ser ejecutado.
- El núcleo del SO debe residir también en memoria para que pueda gestionar las llamadas a sistema del proceso, interrupciones de temporizador así como las posibles excepciones.

Típicamente puede haber múltiples (decenas!) de procesos ejecutante

- El sistema operativo debe configurar el hardware para que cada proceso sólo pueda acceder a su parcela de memoria.
- Si un proceso intenta acceder (por lectura o escritura) a una posición de memoria no permitida, se produce también una excepción.

Protección de memoria

¿Cómo puede evitar un SO (en particular, el núcleo) que un proceso acceda a partes no permitidas en memoria física? De nuevo, se hace gracias al hardware.

A lo largo de la historia ha habido diversas formas de enfocar el problema.

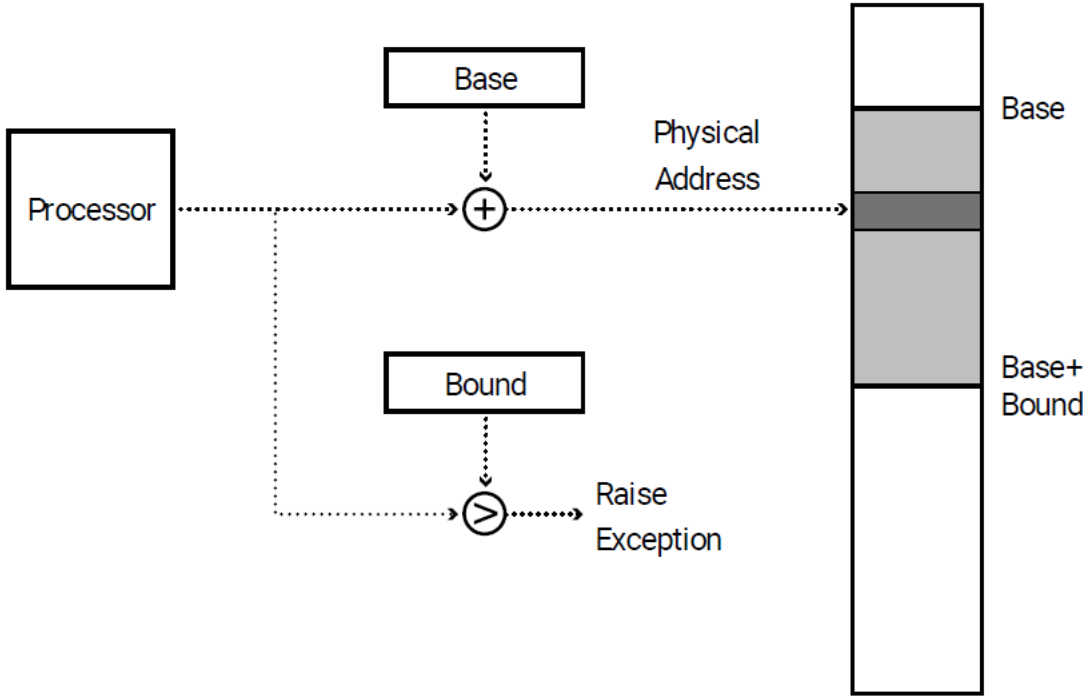
- Un esquema sencillo utiliza dos registros hardware: **base y límite**.
- Hoy en día los ordenadores de sobremesa utilizan **memoria virtual**.

Los explicaremos brevemente, lo veremos con más detalle en el capítulo de memoria virtual.

Protección de memoria: base y límite

Estos registros solo pueden ser modificados por instrucciones privilegiadas

- Cada vez que se accede a la memoria, se suma la **base** a la dirección. Si se supera el límite (**bound**), se produce una excepción
- El SO opera directamente sobre la memoria real.



Protección de memoria: base y límite

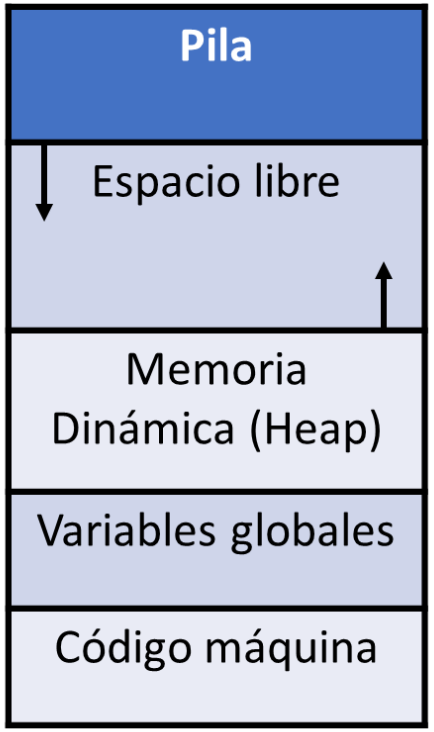
El esquema con dos registros (base y límite) tiene importante defectos:

- **¿Pila y memoria dinámica expansible?** Con solo 2 registros se debe prever la memoria máxima que ocupará el proceso en el momento de cargarlo de disco a memoria.
- **¿Compartición de memoria?** Este esquema no permite que diversos procesos compartan una zona de memoria (p.ej. Instrucciones máquina)
- **¿Fragmentación de memoria?** Con este esquema todos los procesos ocupan un espacio continuo de memoria que no se puede fragmentar en trozos.

Protección de memoria: Memoria virtual

- La memoria virtual se utiliza en la mayoría de los procesadores modernos.
- El procesador genera **direcciones virtuales** que son traducidas por un hardware específico a una dirección física

Primer bit: Indica si la página esta presente al espacio de direcciones
Cuatro bits: Permite la traducción entre direcciones del espacio virtual y físico



Estructura de un proceso en memoria

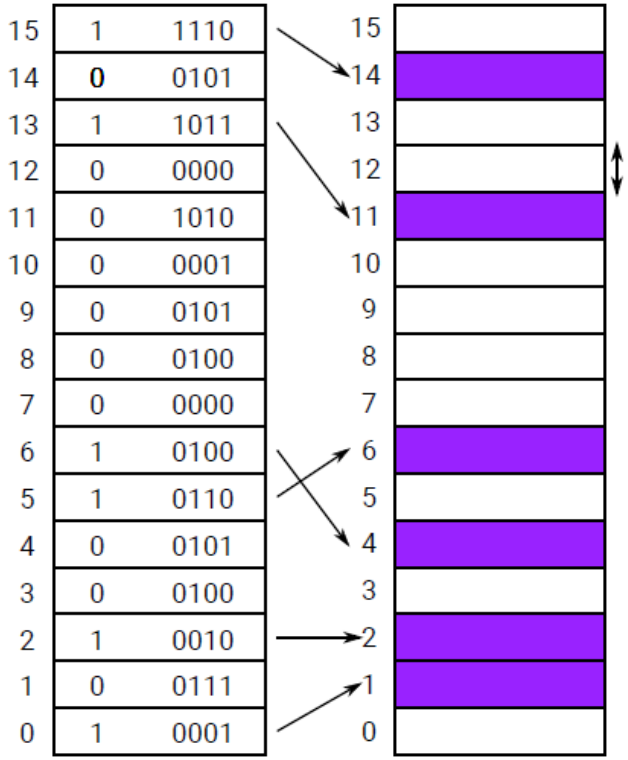


Tabla de páginas del proceso

Espacio de direcciones físicas

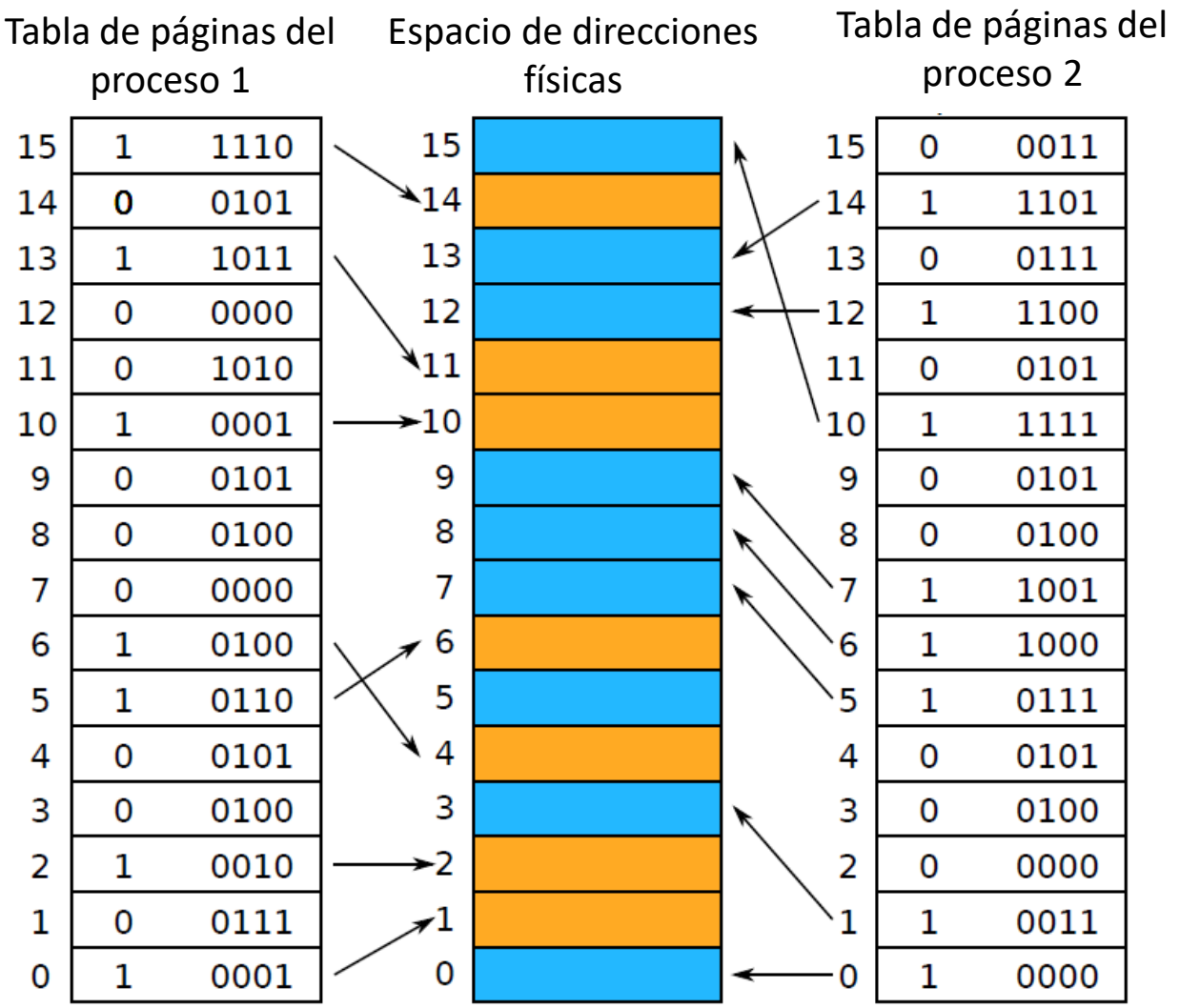
Protección de memoria: Memoria virtual

Características:

- Los **procesos viven al espacio virtual**. Cada proceso cree tener disponible todo el espacio de memoria posible, aunque no exista físicamente.
- El **SO (núcleo) se encarga de gestionar las tablas de procesos**.
- El mapeado de una dirección virtual a una dirección física se realiza a nivel de hardware.
- Posiciones contiguas a memoria virtual no tienen por qué ser continuas en el espacio físico.

Protección de memoria: Memoria virtual

Ejemplo de dos procesos. La memoria virtual permite gestionar la protección de memoria entre estos.



Modo de operación dual

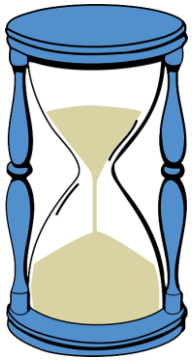
- ¿Qué hardware es necesario para que el núcleo pueda proteger los procesos entre sí?
  - **Instrucciones privilegiadas:** Todas las instrucciones potencialmente peligrosas están prohibidas en modo usuario.
  - **Protección de memoria**: Todos los accesos a memoria física de un proceso fuera del espacio que se le ha asignado están prohibidos en modo usuario.
  - **Interrupciones de temporizador**: Independientemente de lo que haga un proceso, el núcleo del proceso debe tomar el control de forma periódica de la máquina para realizar "gestiones".
- Además, el hardware debe proveer de un mecanismo para pasar de forma segura de modo núcleo a modo usuario ya la inversa.

Interrupciones de temporizador

- Un proceso que se ejecuta en un SO puede ejecutar cualquier instrucción no privilegiada, llamar cualquier función, hacer un bucle, etc.
- El SO hace creer a cada proceso que tiene toda la CPU disponible para si mismo. Para ello, el SO asigna de forma periódica el uso de la CPU a cada proceso, de tal manera que todos los procesos tienen su tiempo de ejecución.
- Si el proceso se cuelga en un bucle infinito, ¿Cómo se puede parar? El SO comprueba periódicamente si el usuario quiere para el proceso.
- Para realizar todas estas gestiones, **el SO toma de forma periódica el control de la CPU**

Interrupciones de temporizador

- El SO utiliza un temporizador a nivel de hardware que interrumpe la ejecución del proceso en determinados momentos (p.ej. 10 ms)



- Al llegar el temporizador a 0, se transfiere el control a una determinada función del SO y se pasa a modo núcleo. El SO puede entonces decidir que proceso se ejecuta a continuación en la CPU, accede a disco, red, etc.
- Se pueden producir otras interrupciones de ejecución de un proceso: el disco avisa al SO que tiene que leer datos, el ratón avisa que se ha movido, etc.

Modo dual de operación: modo núcleo/usuario

- El hardware ofrece una serie de servicios para que el SO (núcleo) pueda gestionar la ejecución de procesos. Existe **una estrecha colaboración entre hardware y núcleo**.
- El sistema x86 soporta más modos de operación (en concreto, 4). La idea detrás es permitir que el SO pueda ejecutar utilizando múltiples niveles de privilegio para asegurar que un error en una determinada parte no provoque un fallo del sistema. Actualmente ni Linux, ni MacOS, ni Windows hacen servir a los 4 niveles.
- En la actualidad, en algunos SOs, el núcleo del SO ejecuta en modo núcleo mientras que otra porción del SO en modo usuario (por seguridad). Esto depende de la arquitectura del SO (lo veremos más adelante).

Gracias

El núcleo: Transferencia de control seguridad entre usuario y núcleo

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Tranferéncia de control de seguridad

- Todos los procesos se ejecutan en modo restringido (usuario) con la protección necesaria.
- ¿Cómo se pasa de forma segura del modo usuario al modo núcleo (y viceversa)?
- Cada proceso realice varias transiciones del modo usuario al modo núcleo (y viceversa).
- Estas transiciones entre los diferentes modos son muy habituales (100s-1000s). Por tanto deben ser rápidas y fiables.

Tranferéncia de control de seguridad

Los SOs modernos utilizan diversas técnicas de optimización para reducir el número de transiciones necesarias para realizar una tarea determinada

- **Memoria compartida**: Permite que varios procesos accedan a la misma zona de memoria sin tener que cambiar al modo núcleo.
- **Mapeo de archivos**: Permite que los archivos se muestren directamente en la memoria de un proceso sin tener que leer o escribir explícitamente en el archivo.
- **Temporizadores**: Permiten que los procesos realicen operaciones de espera sin tener que cambiar al modo núcleo.
- **Protocolos de comunicación eficientes**: Protocolos como sockets y tuberías minimizan el número de transiciones entre el modo usuario y el modo núcleo.
- **Planificación de procesos**: Optimiza la manera en que los procesos se ejecutan en el SO, lo que puede reducir la necesidad de realizar cambios de modo para cambiar entre diferentes procesos.

Transferencia modo usuario a modo núcleo

- ¿Cuándo se realiza la transferencia de modo usuario a modo núcleo?

Existen 3 escenarios:

  - **Excepciones**
  - **Interrupciones**
  - **Llamadas a sistema** (cridas a sistema)

- Las excepciones y llamadas a sistema son síncronas (se producen por la ejecución del proceso)
- Las interrupciones son asíncronas

Modo usuario a modo núcleo: excepciones

- **Excepción**: “Evento inesperado durante la ejecución de un proceso”
  - El proceso intenta ejecutar una instrucción privilegiada
  - El proceso intenta acceder a una posición de memoria fuera del espacio permitido.
  - Acceso de escritura a una posición de memoria con solo permisos de lectura
  - División por cero
  - “Breakpoint” de un debugger
  - Etc.
- Al producirse una excepción, el hardware para la ejecución del proceso actual y pasa a ejecutar en modo núcleo una determinada función del núcleo del SO.
- Permite a los procesos detectar y gestionar errores en tiempo de ejecución.

Modo usuario a modo núcleo: excepciones

- Las excepciones son útiles en la **virtualización**: emulan hardware que no existe.
- Por ejemplo, los procesadores de bajo consumo pueden no soportar determinadas operaciones de coma flotante. El SO puede simular estas operaciones.
  - Cuando un proceso realiza una operación de este tipo se produce una excepción.
  - El SO emula la instrucción.
  - El SO devuelve el valor al proceso y continua su ejecución.

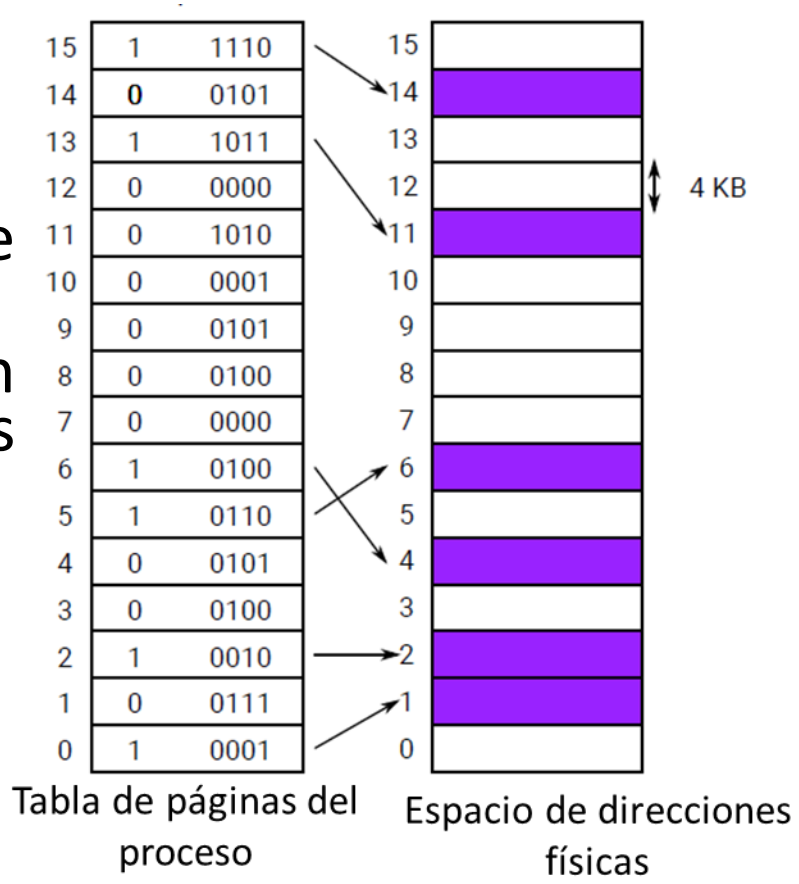
Modo usuario a modo núcleo: excepciones

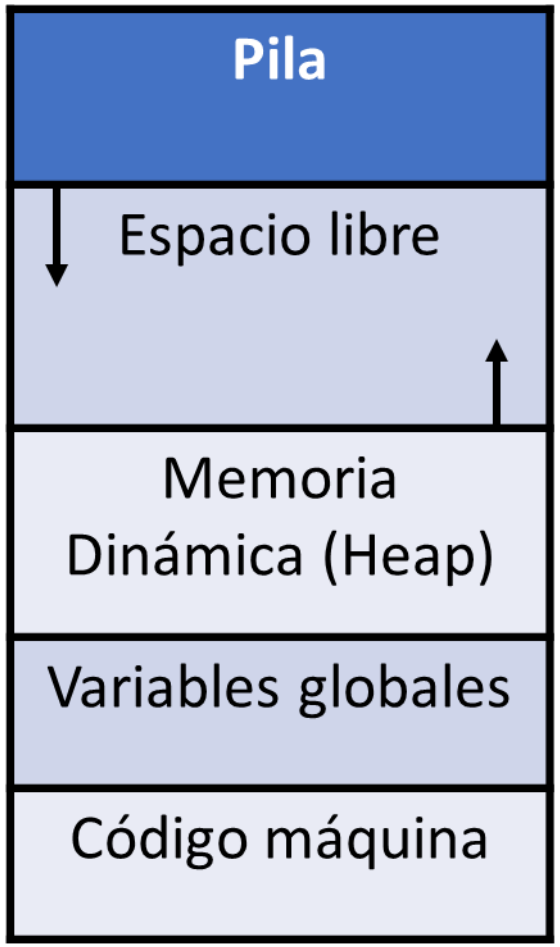
- ¿Qué pasa cuando ejecutamos la máquina virtual de la asignatura?

- ¿En que modo se ejecutan las instrucciones dentro de la máquina virtual?
- La máquina virtual (p.ej. VirtualBox) es un proceso de usuario del SO host (el que ejecuta realmente la máquina)
- Cuando un SO se ejecuta en modo usuario a una máquina virtual, ésta intentará acceder al hardware como si tuviera privilegios totales.
- El SO host capturará esta excepción y realizará la operación solicitada.

Modo usuario a modo núcleo: excepciones

- Las excepciones también se utilizan para la gestión de memoria virtual.
- Se producen cuando:
  - Un proceso accede a una zona de memoria no permitida
  - Un proceso accede a una posición válida pero en ese momento no es a la RAM





Estructura de un proceso en memoria

Modo usuario a modo núcleo: interrupciones

- **Interrupción**: “Señal asíncrona del hardware producida por un evento” (no producido por la ejecución de un proceso)
- Funciona de manera similar a una excepción. El procesador para la ejecución del proceso actual y pasa a ejecutar una determinada función del SO.
- Cada tipo de interrupción tiene una función gestora asociada.
  - Interrupciones por temporizador
  - Interrupciones sobre operaciones E/S de un dispositivo externo (movimiento de ratón, finalización de una operación a disco, llegada de datos por red, etc.)
- Método de comunicación entre dispositivos externos y SO (solicita su atención)

Modo usuario a modo núcleo: llamadas a sistema

- **Llamada a sistema (syscall)**: Cualquier función que provee el SO y que puede ser llamada por el usuario.
- Es la interfaz que ofrece el SO a los procesos para acceder a los dispositivos o realizar otra operación (restringida) que solo tenga acceso el núcleo.
- A nivel de programación se parece a una llamada a una función (la llamada a sistema tiene parámetros y valores de retorno)

Modo usuario a modo núcleo: llamadas a sistema

- Las llamadas a sistema se implementan gracias a una instrucción específica (*trap instruction*)
- **Trap instruction**: instrucción de procesador especiales que se utilizan para implementar las llamadas al sistema
- Pasos:
  1. Cada llamada a sistema tiene un ID único
  2. Al invocar una llamada a sistema, el ID se indica en un registro de proceso específico y se ejecuta la *trap instruction*.
  3. La CPU pasa de modo usuario a modo núcleo
  4. El SO ejecuta el código de la operación solicitada en el núcleo
  5. Una vez finalizada, se vuelve a modo usuario continuando con la ejecución justo después del *trap instruction*.

Modo usuario a modo núcleo: llamadas a sistema

- El SO ofrece un gran número de llamadas a sistema
- Linux ofrece más de 300 que permiten solicitar servicios y recursos del núcleo del SO
- Incluyen operaciones como:
  - Control de procesos (creación, ejecución, finalización,…)
  - Gestión de ficheros (crear, abrir, leer, cerrar, borrar,…)
  - Gestión de dispositivos (conexión, desconexión, escribir, leer, …)
  - Comunicación entre procesos (escribir y enviar mensajes,…)
  - Información de mantenimiento (establecer hora y fecha, listar procesos activos,…)

Modo usuario a modo núcleo: llamadas a sistema

- Listado de llamadas a sistemas:
  - `open()`: abre un archivo y devuelve un descriptor de archivo para su uso posterior.
  - `read()`: lee datos de un archivo o dispositivo y los almacena en un búfer.
  - `write()`: escribe datos en un archivo o dispositivo.
  - `close()`: cierra un archivo o descriptor de archivo previamente abierto.
  - `fork()`: crea un nuevo proceso duplicando el proceso existente.
  - `exec()`: carga un programa en un proceso recién creado y lo ejecuta.
  - `wait()`: espera a que un proceso hijo termine y devuelve su estado de salida.
  - `pipe()`: crea un par de descriptores de archivo conectados por un canal unidireccional.
  - `socket()`: crea un socket para comunicarse a través de la red.
  - `connect()`: conecta un socket a un servidor remoto.
  - `accept()`: acepta una conexión entrante en un socket de escucha.
  - `mmap()`: asigna un archivo o dispositivo en memoria para su acceso directo.
  - `getpid()`: devuelve el ID del proceso actual.
  - `chdir()`: cambia el directorio de trabajo actual.
  - `stat()`: obtiene información sobre un archivo o directorio

Modo usuario a modo núcleo: llamadas a sistema

```
count = write (fd, vector, nbytes)
```

- Ejemplo de llamada a sistema que permite escribir datos en un dispositivo.
- La función `write`, disponible en una librería de usuario, pone los parámetros en el lugar adecuado y realiza la llamada a sistema (con una *trap instruction*)

```
printf ("El valor es %d\n", i);
```

- Esta instrucción genera, en el espacio de usuario, una cadena a imprimir. Una vez generada la cadena, se realiza una llamada al SO con un `write`.

Modo usuario a modo núcleo: llamadas a sistema

- ¿Como podemos rastrear las llamadas a sistema que realiza una aplicación?
- Mediante la instrucción `strace`
- Por ejemplo:
  - `strace Firefox`
  - `strace ./script`
- `strace` es una herramienta que permite a los usuarios obtener una visión detallada de cómo un proceso interactúa con el SO, lo que puede ser muy útil en muchas situaciones diferentes:
  - Depuración de aplicaciones para identificar problemas o errores
  - Investigación de problemas de rendimiento del sistema
  - Verificación de la actividad del sistema durante pruebas de seguridad
  - Análisis de comportamientos sospechosos en un sistema.

Modo núcleo a modo usuario

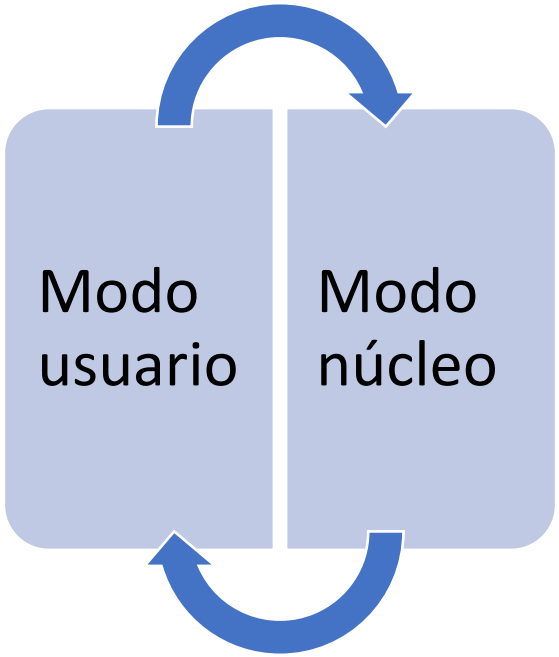
¿Cuándo pasamos del modo núcleo al modo usuario?

- **Al restablecer la ejecución después de una excepción, interrupción o llamada a sistema.** El núcleo acaba de gestionar la operación, restablece la ejecución pasando a modo usuario
- **Al cambiar la ejecución a otro proceso**. El SO decide cambiar la ejecución a otro proceso. Para ello, el SO debe recuperar la información del estado del proceso "antiguo" y restablecerlo en el punto que se va quedar.
- **Ejecutar un proceso nuevo**. Para crear un proceso nuevo, el SO carga en memoria las instrucciones a ejecutar, configura la pila, establece la primera instrucción a ejecutar del proceso, y pasa a modo usuario.
- **Upcall** (llamada del núcleo a los procesos). Mecanismo por el cual los procesos puedan recibir notificaciones asíncronas de eventos, de manera similar a las interrupciones pero a nivel de usuario.

Transferencia de control de seguridad

Hemos visto cuando se transfiere de modo usuario (restringido) a modo núcleo (ilimitado), y viceversa.

Desde el **punto de vista técnico**, ¿cómo se realiza esta transferencia de modos?

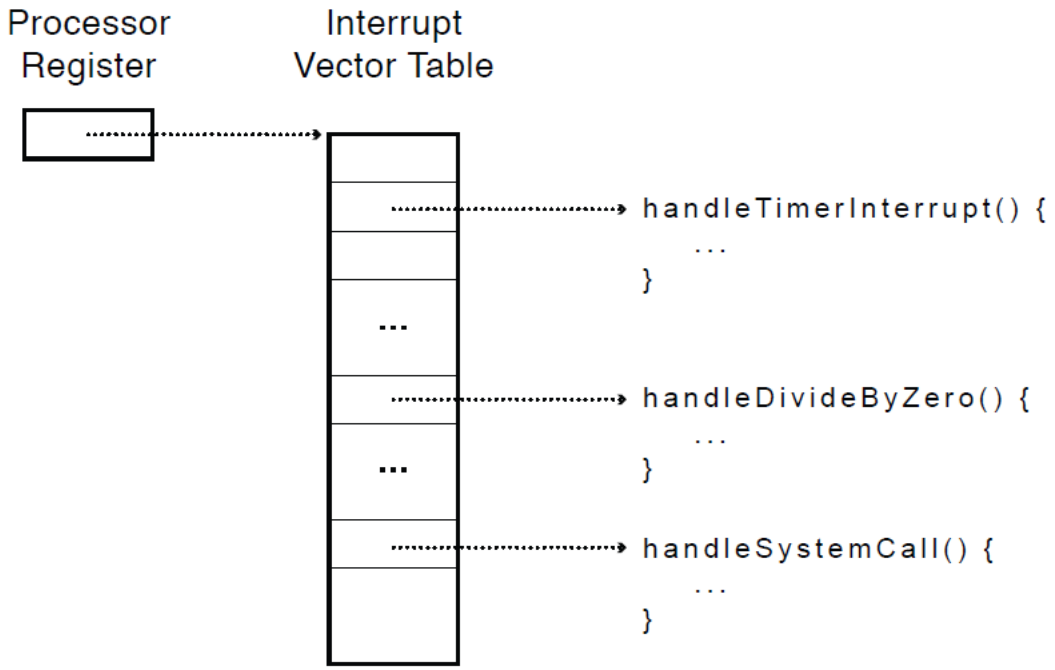


Transferencia de control: implementación técnica

- Esta transferencia se realiza mediante la estrecha colaboración entre el hardware y el núcleo del SO.
- A continuación veremos:
  - La gestión de interrupciones
  - La gestión de excepciones
  - Las llamadas a sistema
- Los mecanismos anteriores se pueden utilizar también para:
  - Ejecutar nuevos procesos
  - Notificaciones asíncronas de eventos a nivel de usuario (*upcall*)

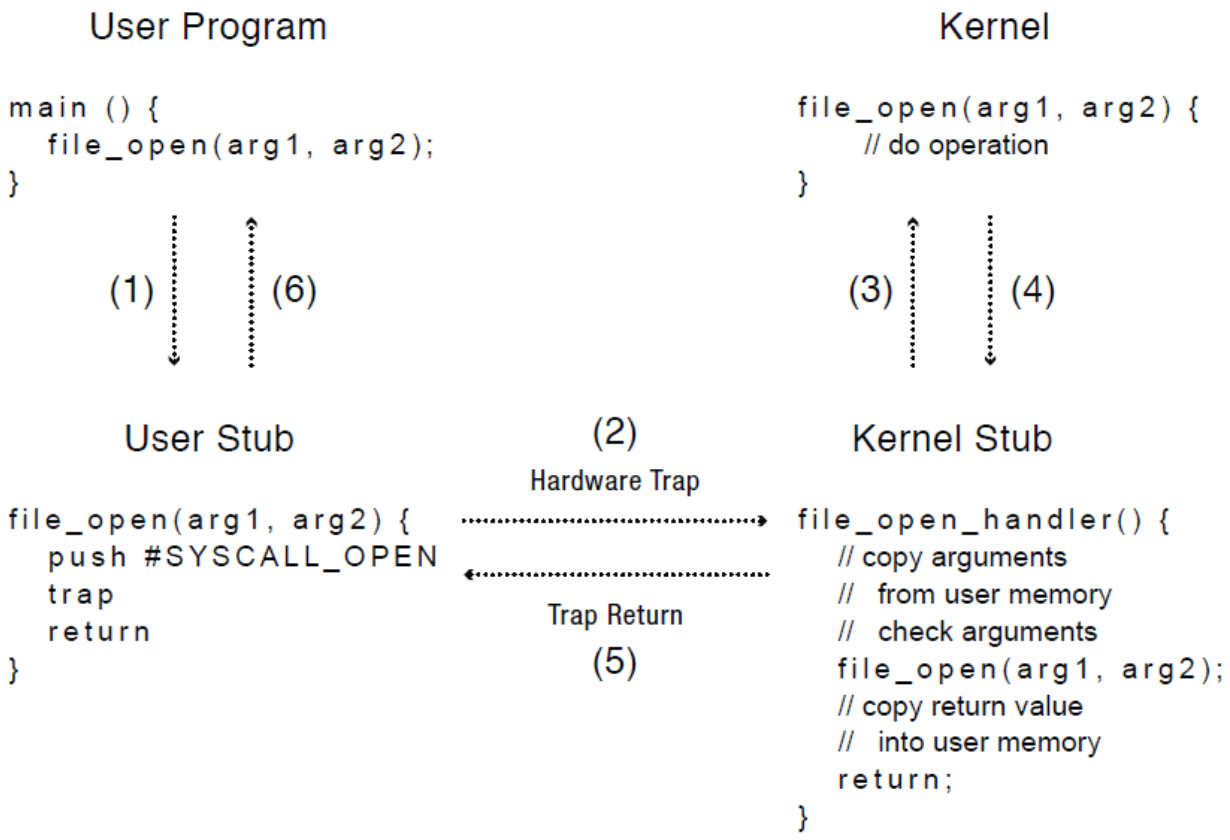
Gestión de interrup., excepciones y llamadas a sistema

- El número entero esta asociado a una entrada en un **vector de interrupciones**.
- Esta entrada apunta a la función que gestionará el evento (interrupción, excepción o llamada a sistema)



Gestión de llamadas a sistema

- En el caso de las **llamadas a sistema**, el proceso es similar a una llamada a cualquier función (a nivel de usuario), pero se transfiere el control al SO.



Gestión de llamadas a sistema

Los pasos a realizar en una **llamada de sistema**:

1. El usuario hace la llamada (`write`). Esta función almacena (en modo usuario) los parámetros de la función a la pila de usuario o registros.
2. Se ejecuta la *trap instruction*. A sistemas x86 también se llama "interrupción del programa". Se pasa a modo núcleo y se ejecuta una determinada función del SO.
3. Dentro del SO, cada llamada de sistema se implementa en funciones diferentes. Se copian los argumentos a la pila del núcleo y se comprueba que tenga valores correctos.
4. Se ejecuta la llamada y, al acabar, se sigue el camino inverso para continuar con la ejecución en el punto en el que se ha realizado la llamada. Se pasa a modo usuario.

Gracias

El núcleo: Estructura de un SO

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Estructura

- El núcleo realiza las tareas básicas del protección entre procesos. ¿Cómo interactúa con el resto de módulos del SO?
- Dependen de la estructura del SO.
- Existen varios tipos de núcleos de SO:
  - Núcleos monolíticos
  - Núcleos híbridos
  - Microkernels

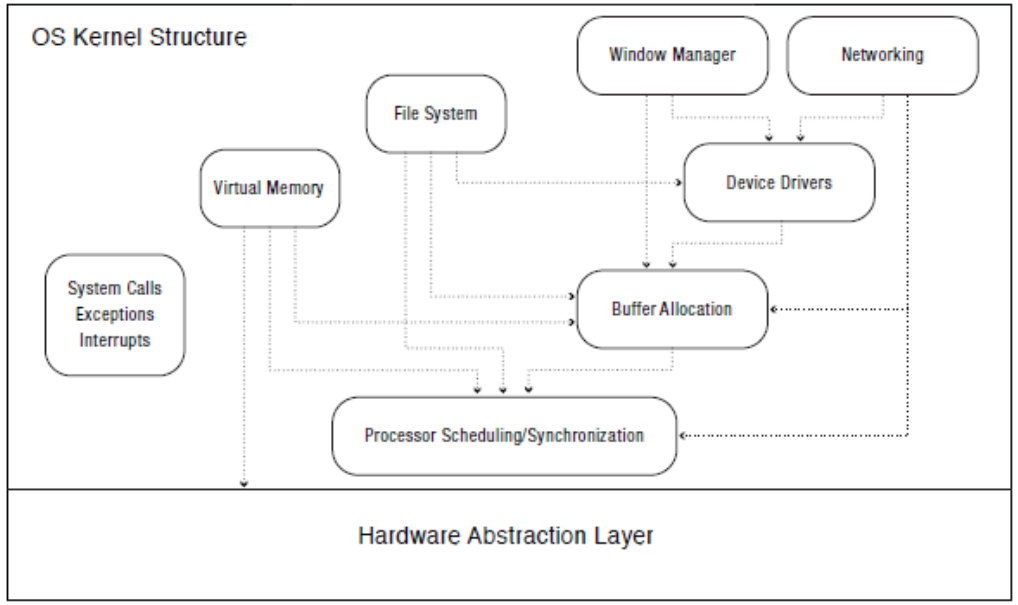
Núcleos monolíticos

- Centralizan la funcionalidad de todo el SO alrededor del núcleo, se aumenta el rendimiento y se facilita la integración total de los módulos. Hay menos flexibilidad para hacer modificaciones.
- Utilizados en la mayoría de los SO actuales (Windows, Linux, MacOS)
- La mayoría de la funcionalidad del SO se centra en el núcleo (por eso normalmente no diferenciamos entre SO o núcleo)
- Los módulos del núcleo llaman directamente a otros núcleos para realizar ciertas tareas.
- La interfaz gráfica, librerías, o líneas de comando se ejecutan en modo usuario. No forman parte del SO.

Núcleos monolíticos

Los diseñadores de SOs pueden desarrollar APIs para comunicar módulos entre si. Existen dos puntos importantes con respecto a la portabilidad:

- ***Hardware Abstraction Layer* (HAL)**: interfaz portable que utiliza el núcleo a las operaciones específicas del dispositivo
- Gestor de dispositivos (***device driver***) dinámico.



Núcleos monolíticos : gestor de dispositivos dinámicos

Es importante que un SO gestione múltiples dispositivos E/S

- Hay una gran variedad de interfaces de hardware para gestionar dispositivos. A Linux, por ejemplo, un 70% del código del SO es software específico para acceder a los dispositivos.
- Es interesante desacoplar el SO de detalles específicos de cada dispositivo. Hoy en día se utilizan gestores de dispositivos dinámicos. Programa específico (programado por el fabricante) que se ejecuta una vez el núcleo ha comenzado a ejecutar y que se encarga de gestionar los dispositivos conectados a la máquina.

Núcleos monolíticos : gestor de dispositivos dinámicos

Los gestores dinámicos tienen un defecto importante:

- Se ejecutan en modo núcleo y por tanto pueden corromper el núcleo del SO y sus estructuras. Un programador malicioso puede utilizar los gestores de dispositivos para introducir un virus.
- Estudios recientes han demostrado que alrededor del 90% de los motivos por los que cuelga el SO son debidos a errores del gestor del dispositivo y no debido al propio SO.
- ¿Qué podemos hacer entonces?

Núcleos monolíticos : gestor de dispositivos dinámicos

- **Gestores de dispositivos ejecutados en modo usuario**: Microsoft i Apple recomiendan que los gestores se ejecuten en modo usuario y que utilicen las llamadas a sistema para acceder a los dispositivos. Esto es cada vez más común, pero muchos códigos de gestores todavía hacen uso de estructuras internas del SO, cosa que dificulta su "traducción" a modo usuario.
- **Gestor de dispositivos en entorno restringido (*sandbox*)**: Se propone que el gestor se ejecute en un entorno restringido (con privilegios limitados) dentro del SO. También pueden ejecutarse en una máquina virtual, pero el rendimiento será más bajo.

Microkernels

- En un núcleo **microkernel**, sólo se ejecutan las funciones esenciales del SO, mientras que las funciones no esenciales, como los controladores de dispositivos y los sistemas de archivos, se ejecutan en el espacio del usuario
- Al descentralizar la funcionalidad del núcleo, se augmenta la seguridad y el mantenimiento del sistema, pero puede reducir la eficiencia debido a la constante comunicación entre el núcleo y espacio de usuario.
- Ejecuta el SO en modo usuario (tanto como sea posible)
- Windows NT se diseñó inicialmente como microkernel. Gran parte de su funcionalidad migró a otro núcleo por motivos de rendimiento. Minux (precursor de Linux), tiene un diseño microkernel, pero Linux no.

Microkernels

- Para el programador no hay diferencia a la hora de programar para un SO monolítico o microkernel.
- El microkernel facilita la modularización, fiabilidad, y seguimiento de errores.
- Por otro lado, reduce el rendimiento ya que no se realizan llamadas directas (como en el monolítico)

Núcleos híbridos

- El **núcleo híbrido** es una combinación de los núcleos monolíticos y microkernel.
- Algunas funciones esenciales del SO se ejecutan en el espacio del núcleo, mientras que otras se ejecutan en el espacio del usuario.
- Esto permite un equilibrio entre la eficiencia y la seguridad del núcleo monolítico y la flexibilidad del núcleo microkernel.
- En la actualidad, se intenta adoptar un modelo híbrido en el que parte de los servicios se ejecutan en modo usuario y parte en modo núcleo. Todo depende del compromiso entre complejidad y rendimiento.

Gracias

El núcleo: Conclusiones

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Conclusiones

- El **núcleo** es el corazón del SO.
- El hardware ofrece una serie de servicios para que el núcleo del SO pueda gestionar la ejecución de **procesos**. Todo ello gracias a una estrecha colaboración entre el hardware y el SO.
- El hardware ofrece **instrucciones privilegiadas**, herramientas de protección de memoria e interrupciones de temporizador.
- Se puede pasar de **modo usuario** a **núcleo** mediante las **excepciones**, **interrupciones** y **llamadas a sistema.**
- Gracias a estas herramientas del hardware, el SO puede hacer el papel de árbitro, ilusionista y ofrecer una interfaz común a los procesos.

Tendencias futuras

- **Ajuste de protección:** Los SOs pueden ofrecer más herramientas y mecanismos para ajustar la protección de los recursos del sistema, como archivos, procesos y memoria. Políticas de seguridad más avanzadas, mejores controles de acceso, mayor capacidad de aislamiento y segmentación de recursos, y mayor protección contra amenazas cibernéticas.
- **Virtualización del hardware**: Técnica que permite ejecutar múltiples SOs en una sola máquina física. Se espera que los SOs sigan mejorando su soporte para la virtualización, lo que puede incluir una mayor eficiencia y rendimiento de las máquinas virtuales, mejores capacidades de migración y gestión de recursos, y mayor soporte para arquitecturas de hardware avanzadas
- **Interoperabilidad:** Mejorar la interoperabilidad con otros sistemas y aplicaciones, lo que puede facilitar la integración y el intercambio de datos entre sistemas heterogéneos (mayor estandarización de los protocolos de comunicación, mejores herramientas de integración y automatización, y un soporte más amplio para los sistemas de virtualización y contenedores).

Gracias